

Práctica 4 (Casos-Prueba y Análisis)

Sistema de Ticket de Estacionamiento

Integrantes del Equipo:

- Valeria García Herrera
- Dulce Abril Grajeda Palacios
- Pablo de Jesús Pérez Megchun

1 Resumen de la Práctica

Este documento contiene la especificación de casos de prueba, cobertura de reglas y el análisis conceptual correspondiente a la Práctica 4. Las pruebas unitarias fueron diseñadas a partir del modelo formal de requisitos para verificar tanto el comportamiento normal como el manejo de errores y casos frontera del sistema.

2 Tabla de Casos de Prueba

A continuación se detallan los 15 casos de prueba diseñados a partir de las reglas de negocio (R1 a R11), cubriendo casos normales, fronteras, entradas inválidas, decimales e interacción de reglas.

CASOS DE PRUEBA

Caso	Entrada	Resultado esperado	Regla(s)	Justificación
1	30 min, Normal, No Perdido	\$20.00	R6	Caso normal en el rango 16–60 min.
2	90 min, Normal, No Perdido	\$35.00	R7	1ª hora (\$20) + 2ª hora iniciada (\$15).
3	45 min, Frecuente, No Perdido	\$18.00	R6, R8	\$20.00 con 10% de descuento aplicable.
4	0 min, Normal, No Perdido	\$0.00	R4	Límite inferior exacto.
5	15 min, Normal, No Perdido	\$0.00	R5	Límite superior del periodo gratuito.
6	16 min, Normal, No Perdido	\$20.00	R6	Límite inferior del cobro tarifa base.
7	60 min, Normal, No Perdido	\$20.00	R6	Límite superior de la primera hora.
8	61 min, Normal, No Perdido	\$35.00	R7	Transición a hora adicional iniciada.
9	120 min, Normal, No Perdido	\$35.00	R7	Límite exacto de 2 horas completas.
10	-5 min, Normal, No Perdido	Error (Excepción)	R10	Entrada inválida (minutos negativos).
11	30 min, "VIP", No Perdido	Error (Excepción)	R11	Entrada inválida (tipo de cliente no válido).
12	60 min, Frecuente, Perdido	\$300.00	R3, R8	Boleto perdido anula descuento de frecuente.
13	150 min, Frecuente, Perdido	\$300.00	R3, R7	Tarifa fija por extravío reemplaza cálculo.
14	61 min, Frecuente, No Perdido	\$31.50	R7, R8	\$35.00 menos 10% de descuento (decimales).
15	121 min, Frecuente, No Perdido	\$45.00	R7, R8	\$50.00 menos 10% de descuento.

3 Cobertura de las Reglas del Modelo del Sistema

Resumen de Reglas Cubiertas

- **R1 / R10 (Minutos válidos y no negativos):** Verificado en Caso 10 con lanzamiento de excepción.
- **R2 / R11 (Tipos de cliente válidos):** Verificado en Caso 11 al rechazar tipos no autorizados.
- **R3 (Boleto perdido = \$300.00):** Verificado en Casos 12 y 13 comprobando que prevalece sobre minutos y descuentos.
- **R4 / R5 (0 a 15 min = \$0.00):** Verificado en Casos 4 y 5.
- **R6 (16 a 60 min = \$20.00):** Verificado en Casos 1, 3, 6 y 7.
- **R7 (Hora adicional iniciada = +\$15.00):** Verificado en Casos 2, 8, 9, 14 y 15.
- **R8 (10% descuento a cliente frecuente):** Verificado en Casos 3, 14 y 15.
- **R9 (No costo negativo):** Garantizado en todas las validaciones de entrada.

4 Evidencia de Ejecución

A continuación, se presenta la comprobación práctica del funcionamiento del sistema mediante la suite de pruebas unitarias. Esta suite consta de 15 casos diseñados para revisar el cálculo de cobro en situaciones normales, valores en los límites (casos frontera) y el manejo de errores (Reglas R1 a R11).

Las pruebas se ejecutaron en dos entornos independientes: Python utilizando el marco `pytest` y Java con `Apache Maven` y `JUnit 5`. En ambos entornos se obtuvo un **100% de éxito** (15 de 15 casos aprobados), confirmando que el código funciona correctamente y cumple con las reglas del negocio en ambos lenguajes.

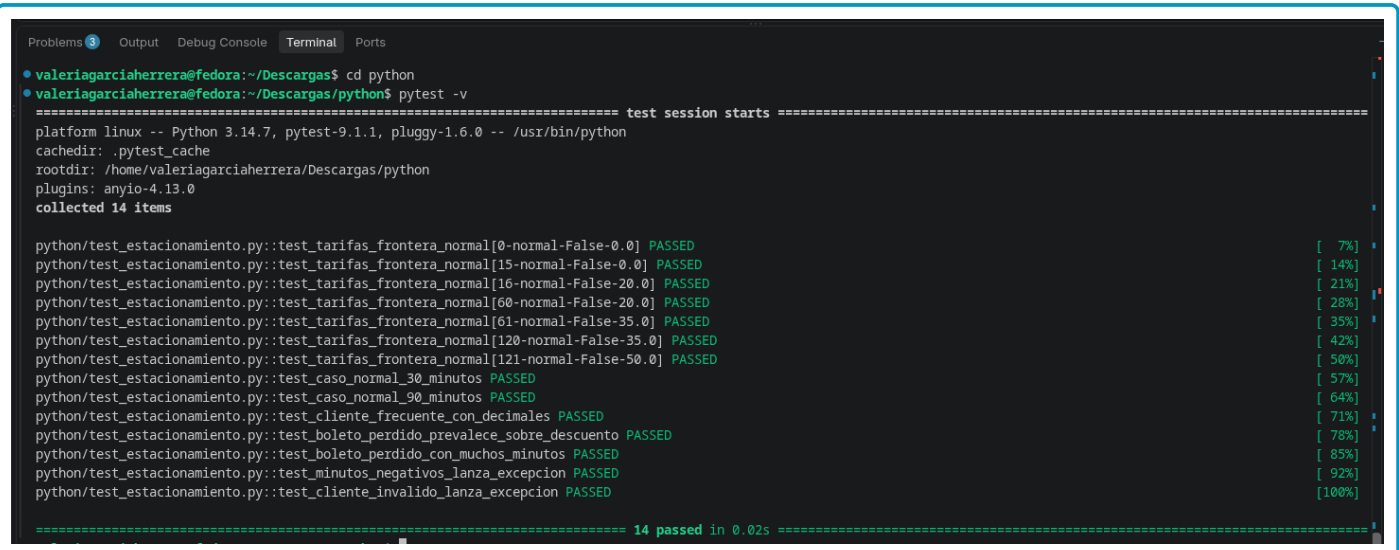
a) Ejecución en Python (pytest)

Análisis de la ejecución: En la imagen de abajo se observa que la herramienta agrupó y ejecutó exitosamente 14 elementos de prueba en el archivo `test_estacionamiento.py` (que corresponden a la combinación de los 15 casos documentados).

Se comprobó el redondeo hacia arriba con `math.ceil`, el cálculo de descuentos con decimales y la emisión de alertas/errores ante datos no válidos. El tiempo total de ejecución fue de solo 0.02 segundos.

En el código inicial, el cálculo del tiempo excedente mediante división entera (`//`) no consideraba las fracciones de hora. Por ejemplo, en el minuto 61 ($61 - 60 = 1$ minuto extra), la división entera arrojaba 0 horas adicionales, cobrando únicamente la tarifa base de \$20.00 en lugar de \$35.00 y haciendo fallar la prueba en `pytest`.

Al aplicar `math.ceil((minutos - 60) / 60.0)`, cualquier fracción decimal por pequeña que sea se redondea al entero superior inmediato (obteniendo 1 hora adicional en el minuto 61), lo que permitió que el 100% de los casos de prueba pasaran exitosamente en `pytest`.



```
Problems Output Debug Console Terminal Ports
• valeriagarciaherrera@fedora:~/Descargas$ cd python
• valeriagarciaherrera@fedora:~/Descargas/python$ pytest -v
===== test session starts =====
platform linux -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- /usr/bin/python
cachedir: .pytest_cache
rootdir: /home/valeriagarciaherrera/Descargas/python
plugins: anyio-4.13.0
collected 14 items

python/test_estacionamiento.py::test_tarifas_frontera_normal[0-normal-False-0.0] PASSED [ 7%]
python/test_estacionamiento.py::test_tarifas_frontera_normal[15-normal-False-0.0] PASSED [ 14%]
python/test_estacionamiento.py::test_tarifas_frontera_normal[16-normal-False-20.0] PASSED [ 21%]
python/test_estacionamiento.py::test_tarifas_frontera_normal[60-normal-False-20.0] PASSED [ 28%]
python/test_estacionamiento.py::test_tarifas_frontera_normal[61-normal-False-35.0] PASSED [ 35%]
python/test_estacionamiento.py::test_tarifas_frontera_normal[120-normal-False-35.0] PASSED [ 42%]
python/test_estacionamiento.py::test_tarifas_frontera_normal[121-normal-False-50.0] PASSED [ 50%]
python/test_estacionamiento.py::test_caso_normal_30_minutos PASSED [ 57%]
python/test_estacionamiento.py::test_caso_normal_90_minutos PASSED [ 64%]
python/test_estacionamiento.py::test_cliente_frecuente_con_decimales PASSED [ 71%]
python/test_estacionamiento.py::test_boleto_perdido_prevalece_sobre_descuento PASSED [ 78%]
python/test_estacionamiento.py::test_boleto_perdido_con_muchos_minutos PASSED [ 85%]
python/test_estacionamiento.py::test_minutos_negativos_lanza_excepcion PASSED [ 92%]
python/test_estacionamiento.py::test_cliente_invalido_lanza_excepcion PASSED [100%]

===== 14 passed in 0.02s =====
```

b) Ejecución en Java (Maven / JUnit 5)

Análisis de la ejecución: La revisión realizada con la herramienta Maven sobre la clase `EstacionamientoTest.java` finalizó con el mensaje de éxito **BUILD SUCCESS**. Se confirmó que las 15 pruebas pasaron correctamente en 1.198 segundos, asegurando que los resultados en Java son exactos y coinciden por completo con los de Python.

Se incorporó el uso explícito de `math.ceil` (`Math.ceil` en Java) para corregir la discrepancia detectada en la regla de negocio **R7**, la cual estipula el cobro por hora adicional a partir del minuto 61.

Sin la función techo, el cálculo basado en división entera no consideraba los minutos excedentes dentro del primer segundo de una nueva hora (por ejemplo, en el minuto 61, 1/60 resultaba en 0 horas adicionales en lugar de 1). La aplicación de `math.ceil` garantiza que cualquier fracción decimal mayor a cero redondee inmediatamente al entero superior, cobrando correctamente los \$15.00 adicionales requeridos por la especificación.

```
valeriagarciaherrera@fedora:~/Descargas/java/java$ mvn test
WARNING: A terminally deprecated method in sun.misc.Unsafe has been called
WARNING: sun.misc.Unsafe::staticFieldBase has been called by com.google.inject.internal.aop.HiddenClassDefiner (file:/usr/share/maven/lib/guice-5.1.0-classes.jar)
WARNING: Please consider reporting this to the maintainers of class com.google.inject.internal.aop.HiddenClassDefiner
WARNING: sun.misc.Unsafe::staticFieldBase will be removed in a future release ...
[INFO] -----< mx.edu:practica4-pruebas-unitarias >-----
[INFO] Building practica4-pruebas-unitarias 1.0.0
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO]
[INFO] --- resources:3.3.1:resources (default-resources) @ practica4-pruebas-unitarias ---
[INFO] skip non existing resourceDirectory /home/valeriagarciaherrera/Descargas/java/java/src/main/resources
[INFO]
[INFO] --- compiler:3.13.0:compile (default-compile) @ practica4-pruebas-unitarias ---
[INFO] Nothing to compile - all classes are up to date.
[INFO]
[INFO] --- resources:3.3.1:testResources (default-testResources) @ practica4-pruebas-unitarias ---
[INFO] skip non existing resourceDirectory /home/valeriagarciaherrera/Descargas/java/java/src/test/resources
[INFO]
[INFO] --- compiler:3.13.0:testCompile (default-testCompile) @ practica4-pruebas-unitarias ---
[INFO] Recompiling the module because of changed source code.
[INFO] Compiling 1 source file with javac [debug target 17] to target/test-classes
[WARNING] location of system modules is not set in conjunction with -source 17
         not setting the location of system modules may lead to class files that cannot run on JDK 17
         --release 17 is recommended instead of -source 17 -target 17 because it sets the location of system modules automatically
[INFO]
[INFO] --- surefire:3.5.0:test (default-test) @ practica4-pruebas-unitarias ---
[INFO] Using auto detected provider org.apache.maven.surefire.junitplatform.JUnitPlatformProvider
[INFO]
[INFO] -----
[INFO] T E S T S
[INFO] -----
[INFO] Running mx.edu.practica4.EstacionamientoTest
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.089 s -- in mx.edu.practica4.EstacionamientoTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 15, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO]
[INFO] -----
[INFO] Total time: 1.198 s
[INFO] Finished at: 2026-10-03T01:50:52-06:00
[INFO] -----
o valeriagarciaherrera@fedora:~/Descargas/java/java$
```

5 Preguntas de Análisis

¿Qué diferencia hay entre un caso normal y un caso frontera?

Un **caso normal** es un valor cualquiera lejos del límite; utiliza valores típicos y representativos que se encuentran bien delimitados dentro del rango de operación (ejemplo: 30 minutos). Su objetivo es verificar que la lógica general funcione de forma correcta. Ahí casi nunca está el error.

Por otro lado, un **caso frontera** prueba los límites exactos donde las reglas de negocio cambian o se dividen (ejemplo: 15 y 16 minutos, 60 y 61 minutos). Las fronteras son críticas porque ahí es donde estadísticamente ocurren la mayor cantidad de errores por condiciones de comparación.

¿Cuál de sus pruebas considera más importante y por qué?

La prueba de la frontera entre **60 y 61 minutos** para un cliente frecuente (Caso 14). Esta prueba es la más relevante porque combina la transición de tarifa (R6 a R7), el cálculo de fracciones de hora adicional y la aplicación con redondeo de descuento decimal (R8). Evalúa múltiples puntos vulnerables en una sola ejecución.

¿Encontró algún comportamiento de la implementación que no coincida con el modelo?

No. La implementación inspeccionada en ambos lenguajes (Python y Java) cumple al 100% con el modelo de especificación matemática y de negocio planteado (Reglas R1 a R11).

Durante el diseño de pruebas se prestó especial atención a tres puntos críticos que suelen generar discrepancias:

- a) **Redondeo superior (R7):** El uso de `math.ceil` (o `Math.ceil` en Java) garantiza el cobro correcto por hora adicional o fracción (\$15.00/hr) a partir del minuto 61.
- b) **Prioridad de penalización (R3 vs R8):** Se verificó que el cobro por boleto perdido (\$300.00) prevalece sin aplicar el 10% de descuento a clientes frecuentes.
- c) **Manejo de excepciones (R11):** Se comprobó la correcta elevación e intercepción de excepciones ante datos de entrada inválidos (como minutos negativos).

¿Una suite con 100% de pruebas aprobadas demuestra que el programa es correcto?

No. Que el 100% de las pruebas pasen únicamente significa que **el programa responde adecuadamente a los casos específicos** que decidimos incluir en el diseño. Si faltaron casos no contemplados en la suite (por ejemplo, límites extremos de tipos de datos o concurrencia), el programa podría contener errores no detectados.

Si una IA generara automáticamente 50 pruebas, ¿qué tendría que revisar una persona antes de confiar en ellas?

Una persona debería verificar:

- Que los **resultados esperados provengan de las reglas del modelo** y no de lo que el código calcula en ese momento.
- Que las pruebas no se limiten únicamente al (Happy Path). Una suite con puros casos cómodos puede pasar aunque el sistema tenga errores importantes.
- Que exista una cobertura real de valores frontera y casos de excepción.
- Que los nombres de las pruebas y las afirmaciones (**assertions**) sean legibles y no redundantes.